

Relatório de Projeto

Sistemas Conexionistas para Aprendizagem

Automática

João Pedro, A103621

Eduarda Ferreira, A106829

Rafael Ferreira, A106819

Conteúdo

Introdução	3
Preparação e Transformação dos Dados	3
Modelos MLP	4
Modelos MLP Single-Stage	4
Modelos MLP Hierárquicos	4
Análise Comparativa e Seleção do Modelo.....	5
Modelos API.....	6
Modelos API Deep Single-Stage.....	6
Análise Comparativa e Seleção do Modelo.....	7
Modelos RL (DQN)	8
Modelos CNN para Séries Temporais	10
Arquiteturas Avaliadas	11
Análise das Curvas de Treino.....	11
Seleção do Modelo CNN.....	13
Modelos RNN (LSTM e GRU).....	13
Análise Comparativa dos Modelos RNN	14
Métricas Globais e Seleção do Modelo	15
Transformers	15
Análise do <i>Loss</i>	16
Análise da <i>Accuracy</i>	16
Análise Comparativa Global e Seleção do Modelo Final	16
Seleção do Modelo Final	17
Conclusão.....	18
Parte B – Modelação e Previsão do Preço de Voos	18
Descrição do problema e objetivos.....	18
Preparação e transformação dos dados	18
Modelo de rede neuronal com embeddings.....	19
Avaliação e comparação dos modelos.....	20

Introdução

A previsão da quantidade de energia solar injetada na rede elétrica constitui um problema relevante no contexto da transição energética, devido à elevada variabilidade temporal associada à produção solar e à sua dependência de fatores meteorológicos. No âmbito da unidade curricular de **Sistemas Conexionistas para Aprendizagem Automática**, este trabalho tem como objetivo desenvolver e comparar diferentes modelos de *Machine Learning* capazes de prever, a cada hora, o nível de energia injetada na rede, utilizando uma escala qualitativa de cinco classes.

Com base em dados reais de produção energética e informação meteorológica, foram exploradas várias abordagens, incluindo modelos tabulares, arquiteturas convolucionais e recorrentes para séries temporais, bem como modelos baseados em atenção. A análise comparativa realizada permitiu identificar o modelo que melhor maximiza a métrica oficial da competição — a *accuracy* — mantendo simultaneamente estabilidade de treino e capacidade de generalização temporal.

Preparação e Transformação dos Dados

Os dados utilizados neste trabalho provêm de duas fontes distintas — dados energéticos e dados meteorológicos — referentes ao período de 2021 a 2022. Atendendo à natureza real dos dados, foi necessário realizar um conjunto de etapas de preparação com o objetivo de garantir consistência, qualidade e adequação à modelação com redes neurais.

A integração das fontes foi efetuada através de uma variável temporal comum (*timestamp*), construída a partir da data e hora nos dados energéticos e da informação temporal nos dados meteorológicos. Esta junção permite incorporar a influência das condições atmosféricas na injeção de energia na rede, enriquecendo o conjunto de dados.

Relativamente à limpeza, procedeu-se à correção de formatos numéricos nas variáveis energéticas medidas em kWh, garantindo a correta conversão para valores numéricos. Nos dados meteorológicos, os valores em falta foram tratados através de uma estratégia gradual: interpolação temporal sempre que possível e, em alternativa, imputação por medianas calculadas por combinação de mês e hora, preservando padrões sazonais e diários.

Foram ainda construídas diversas variáveis temporais a partir do *timestamp*, incluindo dia da semana, mês, hora e indicação de feriado. Para representar corretamente fenómenos periódicos, como a hora do dia e o mês do ano, utilizaram-se codificações cíclicas baseadas em funções seno e cosseno. Adicionalmente, foram definidos segmentos temporais qualitativos (por exemplo, manhã, tarde e noite), bem como variáveis combinadas que capturam interações entre o período horário e a sazonalidade.

As variáveis contínuas foram normalizadas utilizando normalização padrão, assegurando estabilidade no processo de aprendizagem. As variáveis categóricas foram codificadas de forma adequada, garantindo um

tratamento consistente de categorias raras ou ausentes. A variável-alvo foi formulada como um problema de classificação multi-classe e codificada numericamente.

Por fim, os dados foram divididos em conjuntos de treino e validação de forma temporal, utilizando aproximadamente 80% dos dados para treino e 20% para validação. Esta abordagem evita *data leakage* e permite avaliar o desempenho do modelo num cenário realista de previsão futura.

Modelos MLP

Foram avaliadas várias arquiteturas baseadas em *Multilayer Perceptrons (MLP)* para classificação multiclasse da injeção de energia na rede. O objetivo foi analisar o impacto de diferentes estratégias de tratamento do desbalanceamento e de regularização, bem como explorar abordagens alternativas à previsão direta das classes.

Modelos MLP Single-Stage

Inicialmente, foi implementado um **MLP baseline**, utilizado como referência. A partir deste modelo base, foram testadas sucessivas variações com o objetivo de melhorar o desempenho nas classes minoritárias e a capacidade de generalização.

As estratégias exploradas incluem:

- utilização de **pesos de classe**, tanto automáticos como manuais (*soft weights*);
- substituição da função de perda por **Focal Loss**, de forma a enfatizar exemplos difíceis;
- introdução de **regularização explícita**, através de Dropout, penalização L2 e Batch Normalization.

Estas variantes permitem analisar o impacto isolado de cada técnica, mantendo a arquitetura base comparável entre modelos.

Modelos MLP Hierárquicos

Para lidar explicitamente com a forte dominância da classe *None*, foi também explorada uma abordagem **hierárquica em duas etapas**. Nesta formulação, o problema é decomposto numa primeira etapa binária (*None vs Energia*) e numa segunda etapa multiclasse (quatro níveis de energia).

Adicionalmente, foi testada uma variante com **ajuste do limiar de decisão (threshold tuning)** na etapa binária, permitindo calibrar o compromisso entre falsos positivos e falsos negativos e controlar a propagação de erros para a segunda etapa.

Análise Comparativa e Seleção do Modelo

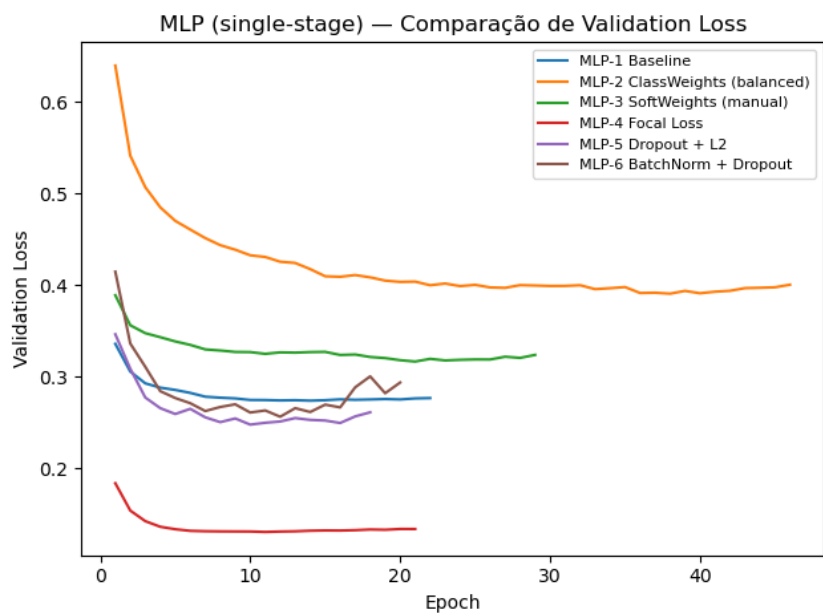


FIGURA 1

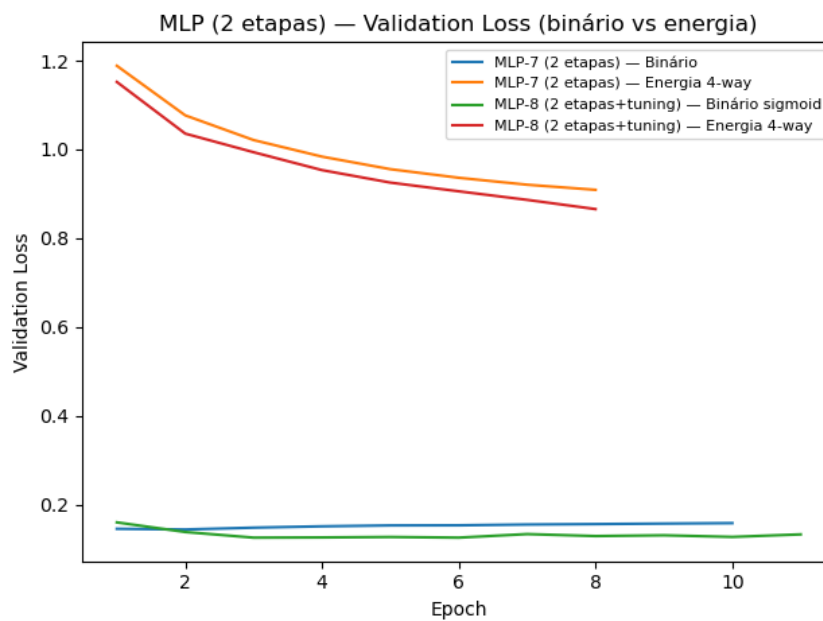


FIGURA 2

A Figura 1 apresenta a comparação da *validation loss* dos modelos MLP single-stage, enquanto a Figura B mostra o comportamento dos modelos hierárquicos. Observa-se que o modelo baseline apresenta convergência estável, mas limitações claras na modelação das classes minoritárias. A introdução de *class weights* conduz a melhorias pontuais, embora com ganhos globais reduzidos.

A utilização de **Focal Loss** demonstra excelente capacidade de generalização, com *validation loss* consistentemente baixa, mas sem maximizar o desempenho médio. Em contraste, os modelos **regularizados** apresentam o melhor compromisso global, com curvas de treino e validação muito próximas e ausência de overfitting significativo. Entre estes, o **MLP com Dropout e regularização L2** destaca-se pelo equilíbrio entre estabilidade, desempenho médio e simplicidade arquitetural.

A abordagem em duas etapas apresenta resultados muito sólidos na classificação binária, com *accuracy* superior a 95%, e melhorias claras na classificação energética quando combinada com *threshold tuning*. No entanto, esta estratégia implica maior complexidade operacional e dependência da calibração do limiar de decisão.

Comparando diretamente o melhor modelo single-stage com a melhor abordagem hierárquica, conclui-se que, apesar das vantagens conceptuais da decomposição em duas etapas, o **MLP single-stage com Dropout + L2** oferece o **melhor compromisso global** entre desempenho quantitativo, robustez do treino e simplicidade.

Assim, este modelo é seleccionado como o **modelo final da secção MLP**, sendo utilizado como referência nas comparações subsequentes com arquiteturas mais avançada

Modelos API

Foram avaliadas várias arquiteturas de redes neuronais implementadas através das diferentes APIs do TensorFlow/Keras, nomeadamente **Sequential API**, **Functional API** e **Subclassing API**. O objetivo foi analisar o impacto da profundidade da rede, da introdução de ligações residuais e do uso de *class weights* suaves no desempenho da classificação multiclasse da injeção de energia na rede.

Modelos API Deep Single-Stage

Numa primeira fase, foi implementado um modelo profundo recorrendo à **Sequential API**, servindo como referência dentro do conjunto de modelos API.

A partir deste, foram exploradas arquiteturas mais flexíveis com a **Functional API**, permitindo a introdução de ligações residuais, bem como variantes com *soft class weights* para lidar explicitamente com o desbalanceamento das classes. Adicionalmente, foi testada uma implementação equivalente recorrendo à **Subclassing API**, com o

objetivo de avaliar se o maior grau de flexibilidade oferecido por esta abordagem se traduziria em ganhos de desempenho.

Análise Comparativa e Seleção do Modelo

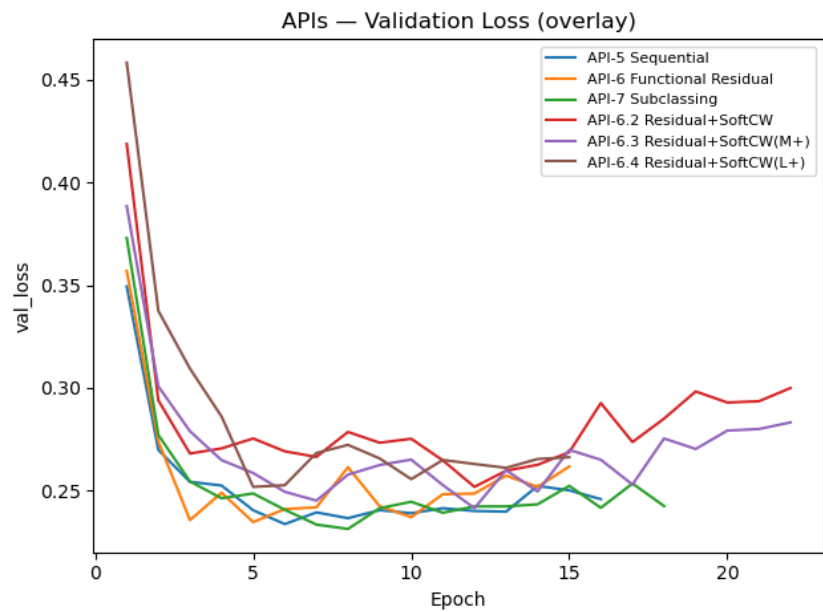


FIGURA 3

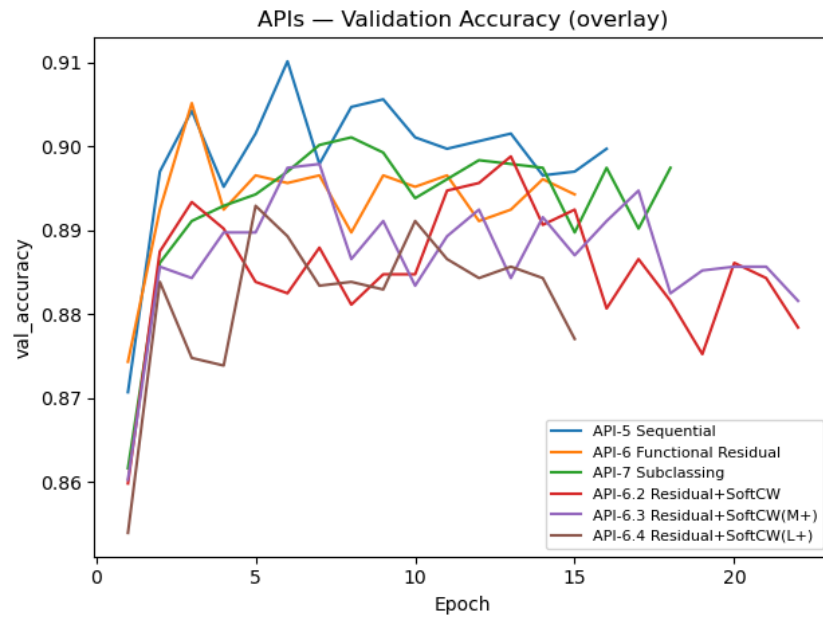


FIGURA 4

A Figura 3 apresenta a comparação da *validation loss* dos diferentes modelos API, enquanto a Figura 4 mostra a evolução da *validation accuracy*. Observa-se que todos os modelos convergem rapidamente, evidenciando processos de treino estáveis e eficazes.

O modelo baseado na **Sequential API** destaca-se por apresentar uma *validation loss* baixa e estável, aliada aos valores mais elevados de *validation accuracy* ao longo do treino. As arquiteturas com ligações residuais demonstram boa estabilidade, confirmando a utilidade das *skip connections* em redes mais profundas, embora sem ganhos claros face ao modelo sequencial.

As variantes com *soft class weights* apresentam melhorias moderadas no desempenho das classes minoritárias, refletidas em valores ligeiramente superiores de *recall macro*.

No entanto, estas melhorias não se traduzem numa vantagem global significativa, sendo acompanhadas por uma ligeira redução da *accuracy* média e maior sensibilidade à calibração dos pesos.

O modelo implementado com a **Subclassing API** apresenta desempenho consistente, mas não evidencia benefícios quantitativos que justifiquem a maior complexidade de implementação quando comparado com as restantes abordagens.

Considerando conjuntamente o desempenho quantitativo, a estabilidade do treino e a simplicidade arquitetural, o **modelo API implementado com a Sequential API (Deep MLP)** apresenta o melhor compromisso global.

Modelos RL (DQN)

Nesta secção foram exploradas abordagens de *Reinforcement Learning* (RL) com **Deep Q-Networks (DQN)**, enquadrando o problema de classificação como uma tarefa sequencial de decisão: a cada instante o agente observa o estado (features) e escolhe uma ação correspondente à classe prevista (*High*, *Low*, *Medium*, *None*, *Very_High*).

O objetivo foi avaliar se um agente treinado por recompensas consegue melhorar a identificação das classes minoritárias, sobretudo quando é incorporado **contexto temporal** (janela de observações).

Modelo RL-9 - DQN com estado instantâneo (tabular)

O RL-9 utiliza um estado **instantâneo** (vetor de features do instante atual) e aprende uma política por *epsilon-greedy*, com *replay buffer* e rede-alvo (*target network*). A recompensa é positiva quando a classe está correta e negativa caso contrário (com *reward shaping* opcional por classe), mas o agente não observa explicitamente contexto temporal — cada decisão depende apenas do instante.

Modelo RL-9.1 - DQN com estado sequencial (window=24) + CNN1D

O RL-9.1 introduz **estado sequencial**, construindo uma janela temporal (por exemplo, **24 passos**) como observação. Para processar esta sequência, a rede Q usa **CNN1D**, permitindo capturar padrões locais no tempo e dependências curtas entre instantes. Esta alteração é relevante porque o fenómeno de injeção de energia tende a apresentar estrutura temporal, e uma janela pode reduzir decisões “miopes” típicas do estado instantâneo.

Análise Comparativa e Seleção do Modelo

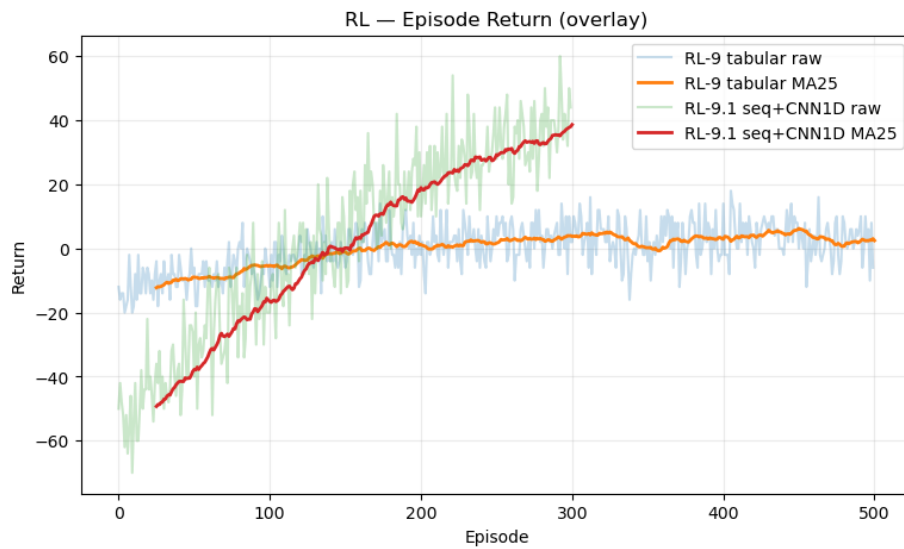


FIGURA 5

A comparação do *episode return* evidencia uma diferença clara entre os dois agentes. O **RL-9** apresenta retornos baixos e instáveis, com ganhos marginais ao longo do treino. Em contraste, o **RL-9.1** mostra uma trajetória de aprendizagem muito mais consistente: a média móvel (MA25) cresce de forma sustentada, indicando que o agente aprende uma política significativamente mais eficaz quando dispõe de contexto temporal.

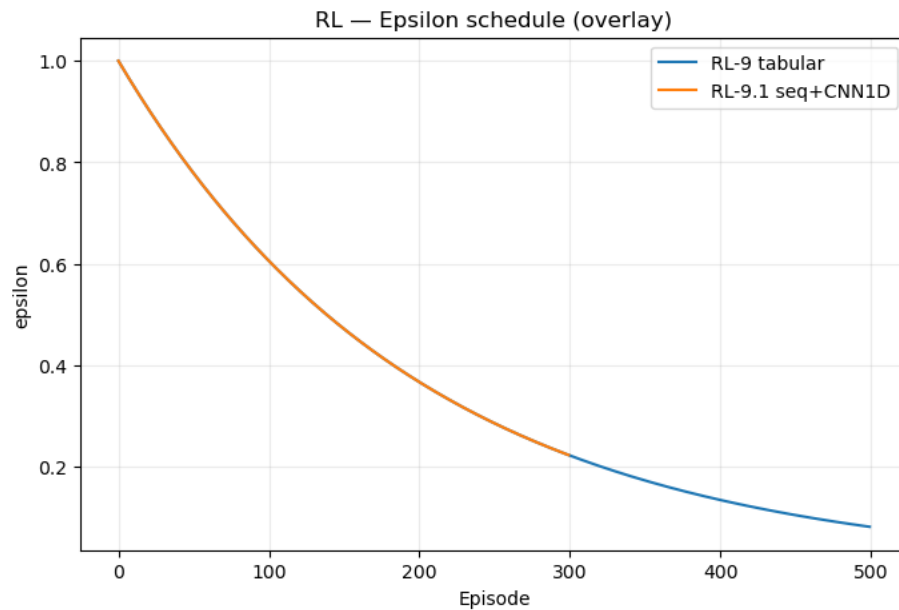


FIGURA 6

O decaimento do epsilon é semelhante entre modelos, garantindo condições de exploração comparáveis. Assim, a melhoria do RL-9.1 não resulta de uma estratégia de exploração diferente, mas sim da **representação de estado mais rica** (sequência) e da capacidade da CNN1D em extrair sinal temporal útil.

Matriz de confusão e métricas. Em validação, o RL-9.1 alcança **accuracy** ≈ 0.85 e **macro-F1** ≈ 0.50 , superando largamente o RL-9 (accuracy ≈ 0.60 ; macro-F1 ≈ 0.27). Esta diferença confirma que o modelo com janela temporal não só melhora o desempenho global, como também consegue um equilíbrio superior entre classes. Observa-se, ainda assim, que a classe **None** continua a ser a mais bem capturada (o que é esperado dado o desbalanceamento), mas o RL-9.1 revela ganhos claros nas classes energéticas (*High/Medium/Very_High*) face ao RL-9, que tende a colapsar muitas decisões para padrões inconsistentes e com baixa estabilidade.

Modelo selecionado na secção RL

Considerando (i) a evolução do retorno durante o treino, (ii) a estabilidade observada no *overlay* de *episode returns*, e (iii) a melhoria substancial nas métricas de validação, o **RL-9.1 (DQN com window=24 + CNN1D)** é selecionado como o melhor modelo desta secção de RL.

Modelos CNN para Séries Temporais

Após a avaliação dos modelos baseados em MLP e Reinforcement Learning, foram exploradas arquiteturas **Convolutional Neural Networks (CNN)** adaptadas a séries temporais, com o objetivo de capturar dependências locais ao longo do tempo através de janelas deslizantes de 24 horas. Estas abordagens

permitem modelar padrões temporais curtos que não são explicitamente considerados nos modelos puramente tabulares.

Arquiteturas Avaliadas

Foram testadas três variantes principais de CNN 1D:

- **CNN-10 (PL9 CNN1D)**: modelo base com convoluções temporais e pooling, sem tratamento explícito do desbalanceamento;
- **CNN-10.1 (CNN1D + Soft Class Weights)**: extensão do modelo base com pesos de classe suaves, visando melhorar o desempenho nas classes minoritárias;
- **CNN-11.1 (CNN1D + Augmentation + Soft Class Weights)**: versão mais avançada, combinando pesos de classe suaves com *data augmentation* temporal leve (jitter e scaling).

Todas as arquiteturas utilizam janelas temporais de 24 observações consecutivas e foram treinadas com *early stopping* baseado na *validation loss*.

Análise das Curvas de Treino

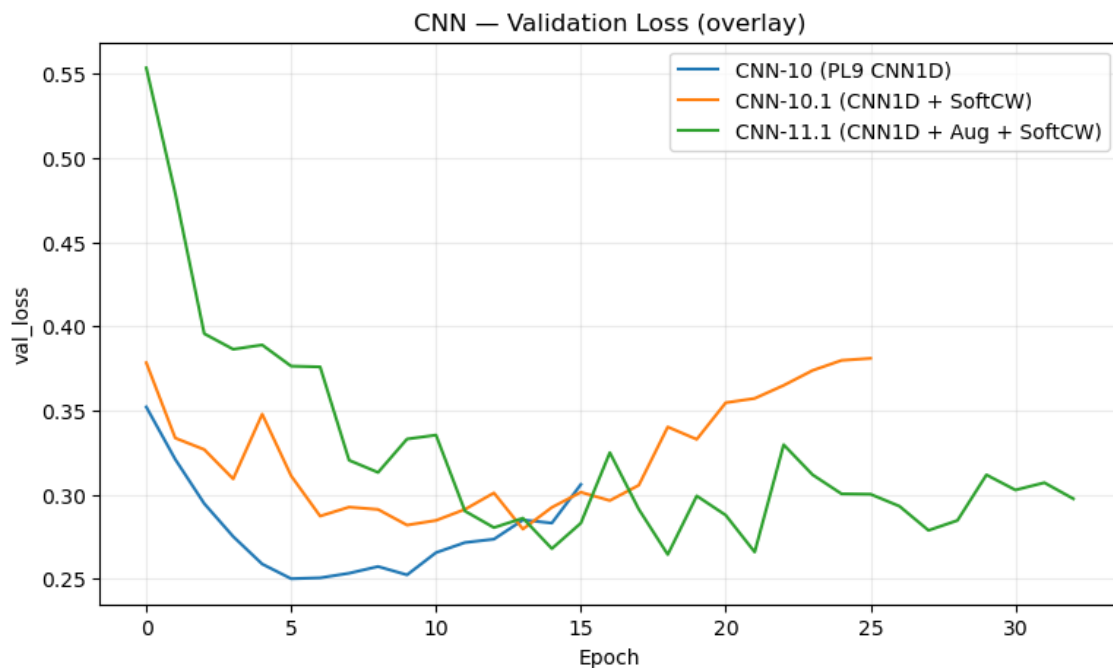


FIGURA 7

A Figura CNN — Validation Loss (overlay) apresenta a evolução da *validation loss* para os três modelos. Observa-se que:

- o **CNN-10** apresenta uma convergência rápida e estável, atingindo o mínimo de *validation loss* por volta da época 6;
- o **CNN-10.1** evidencia uma *validation loss* sistematicamente superior, com tendência crescente após as primeiras épocas, sugerindo menor capacidade de generalização;
- o **CNN-11.1**, apesar de mais complexo, apresenta maior variabilidade ao longo do treino, refletindo o impacto da *data augmentation*, mas sem alcançar valores de *loss* inferiores ao modelo base.

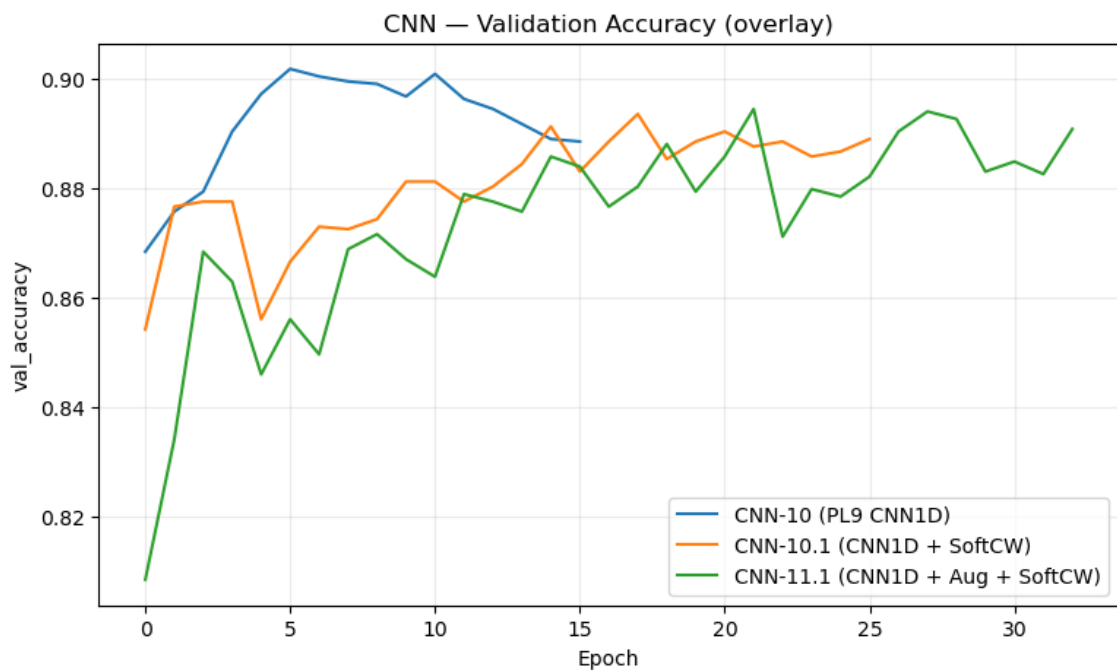


FIGURA 8

Na Figura CNN — **Validation Accuracy (overlay)** verifica-se um comportamento consistente com a análise anterior. O **CNN-10** atinge rapidamente uma *validation accuracy* próxima de 90%, mantendo-se estável ao longo das épocas. Os modelos com *soft class weights* e *augmentation* apresentam melhorias pontuais, mas sem ultrapassar de forma consistente o desempenho do modelo base.

- **CNN-10 (PL9 CNN1D)** obtém o melhor compromisso global, com:
 - *accuracy* $\approx$ **0.902**
 - *macro F1-score* $\approx$ **0.63**
 - *precision macro* $\approx$ **0.68**
 - *recall macro* $\approx$ **0.61**

- **CNN-11.1** apresenta um *recall macro* ligeiramente superior, mas à custa de menor *precision* e *macro F1*, refletindo maior instabilidade introduzida pela *augmentation*;
- **CNN-10.1**, apesar da introdução de *soft class weights*, não consegue superar o modelo base, apresentando simultaneamente menor *accuracy* e *macro F1-score*.

Seleção do Modelo CNN

Com base na análise conjunta das curvas de treino, métricas quantitativas e matrizes de confusão, conclui-se que:

- a introdução de *soft class weights* e *data augmentation* não conduz a ganhos consistentes neste contexto;
- o aumento da complexidade arquitetural não se traduz em melhorias claras de generalização;
- o **CNN-10 (PL9 CNN1D)** apresenta o melhor compromisso entre desempenho, estabilidade e simplicidade.

Assim, o **CNN-10 (PL9 CNN1D)** é selecionado como o **modelo final da família CNN**, sendo utilizado nas comparações globais com os restantes paradigmas explorados na Parte A do trabalho.

Modelos RNN (LSTM e GRU)

Nesta secção foram avaliadas arquiteturas baseadas em **Redes Neurais Recorrentes (RNN)**, com o objetivo de explorar explicitamente a **dependência temporal** dos dados através de janelas sequenciais de 24 horas. Foram testadas duas variantes principais: **LSTM** e **GRU**, ambas treinadas com *soft class weights* para mitigar o desbalanceamento das classes.

Ambos os modelos utilizam:

- janelas temporais de 24 observações consecutivas;
- duas camadas recorrentes empilhadas (96 e 64 unidades);
- regularização por *Dropout*;
- otimização com Adam;
- *early stopping* com base na *validation loss*.

A divisão treino/validação foi feita de forma **temporal**, garantindo coerência cronológica.

Análise Comparativa dos Modelos RNN

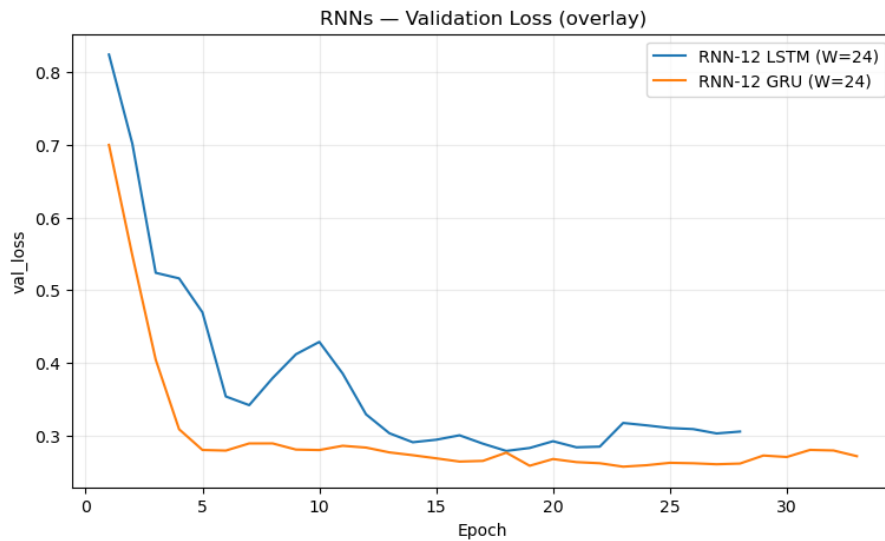


FIGURA 9

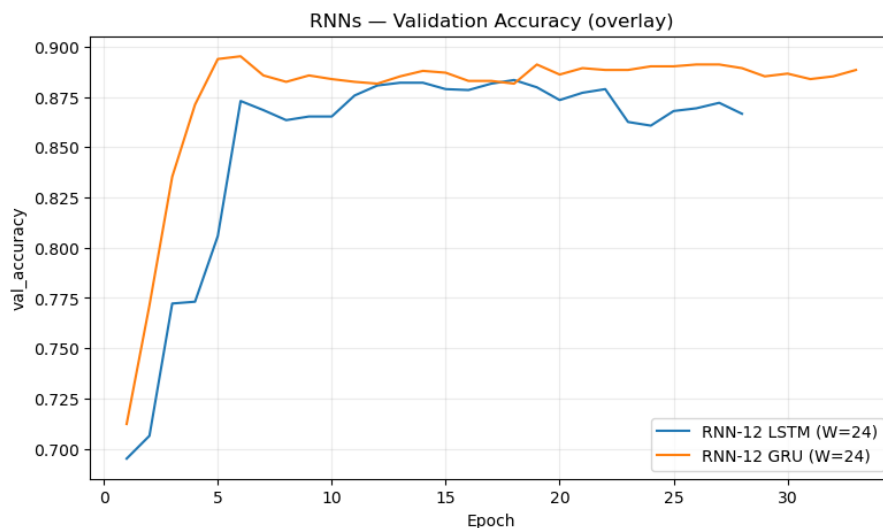


FIGURA 10

Os gráficos de **validation loss** e **validation accuracy** (Figuras 9 e 10) mostram comportamentos distintos entre as duas arquiteturas.

O modelo **LSTM** apresenta uma descida inicial rápida da *validation loss*, mas com maior instabilidade ao longo do treino, evidenciada por oscilações mais pronunciadas e um patamar final de loss superior. A *validation accuracy* estabiliza em torno dos **88%**, sem melhorias consistentes após as primeiras épocas.

Por contraste, o modelo **GRU** revela uma convergência mais **estável e consistente**, com *validation loss* sistematicamente inferior e menos flutuações ao longo das épocas. A *validation accuracy* atinge

valores próximos de **89–90%**, mantendo-se estável durante grande parte do treino, o que sugere melhor generalização.

A análise das **matrizes de confusão** reforça estas observações. Ambos os modelos apresentam desempenho muito elevado na classe dominante (*None*), mas o **GRU** demonstra melhorias claras nas classes minoritárias, em particular:

- maior *recall* nas classes *Low* e *Very_High*;
- melhor equilíbrio global entre *precision* e *recall* nas classes energéticas.

Métricas Globais e Seleção do Modelo

O **F1 macro**, métrica-chave neste problema devido ao forte desbalanceamento das classes, é claramente superior no modelo GRU, indicando um melhor compromisso entre desempenho nas classes maioritárias e minoritárias.

Tendo em conta:

- a maior estabilidade observada nos gráficos de treino e validação;
- o melhor desempenho global em termos de *F1 macro*;
- a melhoria consistente no *recall* das classes minoritárias;

o **modelo RNN-12 GRU (W=24)** é seleccionado como o **modelo representativo da secção RNN**.

Transformers

Modelo 14 (Transformer, W = 24)

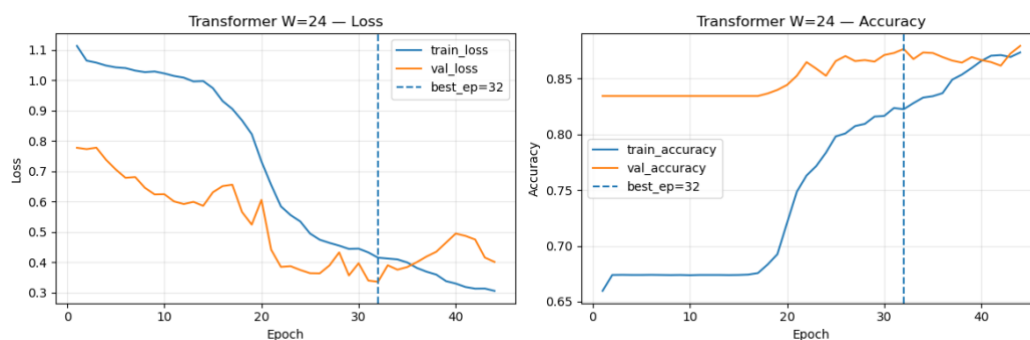


FIGURA 11

A Figura 11 apresenta as curvas de aprendizagem do **Modelo 14 – Transformer temporal**, considerando uma janela temporal de tamanho $W=24$. São representadas as evoluções do *loss* e da *accuracy* nos conjuntos de treino e validação ao longo das épocas.

Análise do *Loss*

Observa-se uma **redução consistente do *training loss*** ao longo das épocas, indicando que o modelo consegue ajustar-se progressivamente aos dados de treino. O *validation loss* apresenta uma descida inicial significativa, atingindo o seu **mínimo por volta da época 32**, após a qual começa a aumentar gradualmente.

Este comportamento indica o início de **overfitting** a partir dessa região, uma vez que o modelo continua a melhorar no conjunto de treino, mas o desempenho no conjunto de validação deixa de beneficiar desse ajuste adicional. Assim, a **época 32** é identificada como a melhor época segundo o critério de minimização do *validation loss*.

Análise da *Accuracy*

Relativamente à *accuracy*, verifica-se um **plateau inicial da *validation accuracy*** nas primeiras épocas, próximo de 0.83. Este comportamento está associado ao **desbalanceamento das classes**, levando o modelo, numa fase inicial, a privilegiar a classe maioritária (“None”).

A partir de aproximadamente 15–20 épocas, observa-se uma **melhoria gradual da *accuracy*** tanto no treino como na validação, refletindo uma aprendizagem mais discriminativa entre as classes. A *validation accuracy* estabiliza em valores próximos de 0.87–0.88, mantendo-se relativamente estável até ao final do treino.

Com base na análise conjunta das curvas de *loss* e *accuracy*, conclui-se que:

- O modelo apresenta **aprendizagem efetiva** ao longo do treino;
- O ponto ótimo de generalização ocorre em torno da **época 32**, coincidindo com o mínimo do *validation loss*;
- Treinar para além desse ponto conduz a **overfitting**, sem ganhos adicionais na generalização;
- O uso de *early stopping* com base no *validation loss* é, assim, **adequado e justificado** para este modelo.

Análise Comparativa Global e Seleção do Modelo Final

Após a fase de exploração, desenvolvimento e otimização de múltiplas abordagens de *Machine Learning*, procedeu-se a uma análise comparativa global dos modelos com melhor desempenho em cada família

arquitetural. Esta análise teve como principal objetivo identificar o modelo que melhor responde aos requisitos do problema proposto, nomeadamente a previsão da quantidade de energia elétrica injetada na rede, numa escala qualitativa, em regime horário, e cuja métrica de avaliação oficial é a *accuracy*.

Dada a natureza temporal do fenómeno em estudo, bem como o forte desbalanceamento entre classes — com clara dominância da classe *None* —, a seleção do modelo final teve em consideração não apenas o desempenho quantitativo, mas também a estabilidade do treino, a capacidade de generalização temporal e a complexidade arquitetural.

Com base nos resultados obtidos, foram selecionados como candidatos finais os seguintes modelos representativos de cada família:

- **MLP Single-Stage com Dropout e regularização L2**, enquanto melhor modelo puramente tabular;
- **Deep MLP implementado com a Sequential API**, representando arquiteturas profundas densas;
- **CNN-10 (PL9 CNN1D) com janela temporal de 24 horas**, enquanto melhor modelo convolucional temporal;
- **RNN-12 (GRU, $W = 24$)**, como melhor arquitetura recorrente;
- **Modelo Transformer temporal ($W = 24$)**, representando arquiteturas baseadas em atenção.

Estes modelos apresentaram desempenho superior e comportamento estável dentro das respetivas categorias, justificando a sua inclusão na comparação final.

Seleção do Modelo Final

Considerando que a métrica oficial de avaliação da competição é a *accuracy*, e que o objetivo principal consiste em maximizar a taxa de acerto global em dados futuros, o modelo **CNN-10 (PL9 CNN1D, $W = 24$)** é selecionado como **modelo final** deste trabalho.

Este modelo oferece o melhor compromisso entre:

- desempenho quantitativo (maior *accuracy* observada);
- estabilidade e robustez do processo de treino;
- capacidade de capturar padrões temporais diários relevantes para a produção solar;
- complexidade arquitetural moderada e facilmente justificável.

Assim, o **CNN-10** é utilizado como modelo de referência para a submissão na competição e para a apresentação final do projeto, por se revelar a solução mais eficaz e alinhada com os objetivos definidos no enunciado.

Conclusão

A comparação entre os diferentes modelos desenvolvidos demonstra que a incorporação explícita de informação temporal é determinante para melhorar o desempenho preditivo na previsão da energia solar injetada na rede. Os modelos temporais superam consistentemente as abordagens puramente tabulares, evidenciando maior capacidade de generalização.

Entre os modelos avaliados, o **CNN-10 (CNN1D, W = 24)** apresentou a melhor *accuracy*, aliada a um treino estável e a uma complexidade moderada, sendo por isso selecionado como modelo final. Esta escolha encontra-se alinhada com os objetivos do projeto e com a métrica de avaliação definida na competição, constituindo a solução mais adequada para a tarefa proposta.

Parte B – Modelação e Previsão do Preço de Voos

Descrição do problema e objetivos

Nesta Parte B aborda-se o problema de **previsão do preço de bilhetes de avião** com base em informação tabular relativa às características do voo.

Este tipo de problema é particularmente relevante no contexto da análise de dados e da aprendizagem automática, dado o carácter não linear e multifatorial da formação de preços no setor da aviação.

O objetivo principal consiste no desenvolvimento de um **modelo de Machine Learning para regressão**, capaz de estimar o preço de um voo (variável alvo contínua *price*) a partir de um conjunto de variáveis explicativas, incluindo a companhia aérea, a cidade de origem e destino, os horários de partida e chegada, o número de escalas, a classe de viagem, a duração do voo e o número de dias de antecedência entre a compra e a data do voo.

Adicionalmente, pretende-se avaliar o impacto das diferentes etapas de **preparação dos dados**, bem como da utilização de **embeddings para variáveis categóricas**, na qualidade das previsões obtidas.

Preparação e transformação dos dados

O ponto de partida para a fase de modelação foi o ficheiro *Clean_Dataset.csv*, que contém um conjunto alargado de registos de voos, incluindo múltiplas variáveis categóricas e numéricas potencialmente relevantes para a explicação do preço do bilhete.

Após a leitura inicial dos dados foi removida uma coluna auxiliar utilizada apenas como índice na origem do dataset.

De seguida, a variável alvo *price* foi convertida para formato numérico, sendo eliminadas todas as observações com valores ausentes ou inválidos, de forma semelhante, a variável *days_left*, que representa o número de dias de antecedência entre a compra do bilhete e a data do voo, foi convertida para formato numérico, removendo-se observações inconsistentes ou incompletas.

Relativamente à duração do voo, verificou-se que os dados originais apresentavam representações diferentes, combinando valores numéricos e descrições textuais. Para resolver este problema, foi definida a função *duration_to_minutes*, responsável por converter tanto valores numéricos (expressos em horas) como strings do tipo “2h 50m” num valor inteiro correspondente à duração total em minutos.

As observações para as quais a conversão não foi possível foram removidas, assegurando consistência na variável temporal sem recorrer a técnicas de imputação artificial.

As variáveis categóricas selecionadas foram inicialmente normalizadas ao nível do formato textual e, posteriormente, codificadas sob a forma de **índices inteiros**, para cada variável categórica foi construído um dicionário de mapeamento entre as categorias observadas e os respetivos índices, reservando-se o índice 0 para uma categoria especial (*UNK*), destinada a representar valores eventualmente não observados durante o treino do modelo.

Esta abordagem permite a utilização de **embeddings** na fase de modelação, evitando o aumento significativo da dimensionalidade que resultaria da aplicação de técnicas tradicionais como *one-hot encoding*, já as variáveis numéricas contínuas foram padronizadas utilizando o método **StandardScaler**.

Com base nestas transformações, foi construído um dataset final, designado por **dataset “gold**, este conjunto de dados foi posteriormente dividido em três subconjuntos distintos: treino (80%), validação (10%) e teste (10%).

Modelo de rede neuronal com embeddings

Cada variável categórica é ligada a uma camada de **embedding de dimensão reduzida**, permitindo transformar categorias discretas em representações vetoriais densas e treináveis.

As variáveis contínuas entram diretamente no modelo através de uma camada de entrada numérica. Todas estas representações são concatenadas e processadas por várias **camadas densas totalmente ligadas**, com funções de ativação do tipo ReLU e aplicação de Dropout como técnica de regularização.

A rede termina numa camada de saída com ativação linear, adequada a problemas de regressão, que produz a estimativa final do preço do voo.

O modelo foi treinado utilizando o otimizador **Adam**, recorrendo ao **erro quadrático médio (MSE)** como função de perda e ao **erro absoluto médio (MAE)** como métrica principal de avaliação do desempenho. Para prevenir overfitting e garantir a seleção automática da melhor versão do modelo, foram utilizados os mecanismos de *Early Stopping* e *ModelCheckpoint*, que interrompem o treino quando o desempenho em validação deixa de melhorar e armazenam o modelo com melhores resultados no conjunto de validação

Avaliação e comparação dos modelos

Após a definição da arquitetura da rede neural, foram treinadas duas variações do modelo com o objetivo de analisar o impacto da regularização por Dropout na qualidade das previsões.

O **Modelo 1** utiliza uma taxa de Dropout de 0.3, enquanto o **Modelo 2** recorre a uma taxa mais elevada de 0.5.

O treino foi realizado durante um máximo de 10 epochs, sendo aplicado os mecanismos já referidos acima, além do *ModelCheckpoint* para armazenar automaticamente o melhor modelo.

Relativamente aos resultados obtidos, o Modelo 1 apresentou um **erro absoluto médio (MAE) no conjunto de validação de aproximadamente 3094 unidades monetárias**, enquanto o Modelo 2 obteve um MAE ligeiramente superior, em torno de **3193**.

Também o erro quadrático médio (MSE) foi inferior no Modelo 1, na ordem dos **27.4 milhões**, em comparação aos **28.9 milhões** observados no Modelo 2.

A análise da evolução das métricas ao longo das épocas indica que ambos os modelos convergem de forma estável, sem sinais evidentes de overfitting severo.

No entanto, o aumento da taxa de Dropout no Modelo 2 conduziu a uma ligeira perda de capacidade preditiva, sugerindo que a regularização aplicada poderá ser excessiva para este conjunto de dados específico.

Deste modo, embora o Modelo 2 apresente maior robustez teórica face ao overfitting, o **Modelo 1 revelou um melhor compromisso entre viés e variância**, alcançando previsões mais precisas no conjunto de validação.

Assim, o Modelo 1 foi selecionado como o modelo final para a tarefa de previsão do preço dos voos.